

# Algorithmes fondamentaux du projet

## Identification d'un réseau de drogue

DIMBIN'NY AINA Nomenjanahary Andrianina Geoffroy

Mai 2026

### Table des matières

|          |                                                              |          |
|----------|--------------------------------------------------------------|----------|
| <b>1</b> | <b>Introduction</b>                                          | <b>2</b> |
| <b>2</b> | <b>Algorithme 1 : Nettoyage du graphe</b>                    | <b>2</b> |
| 2.1      | Objectif . . . . .                                           | 2        |
| 2.2      | Fonction trouver_composantes(G) . . . . .                    | 2        |
| 2.3      | Fonction nettoyage(aretes, N) . . . . .                      | 2        |
| 2.4      | Propriétés du graphe nettoyé . . . . .                       | 2        |
| <b>3</b> | <b>Algorithme 2 : Classification du graphe</b>               | <b>3</b> |
| 3.1      | Coloration gloutonne . . . . .                               | 3        |
| 3.1.1    | Fonction colorier_graphe(G) . . . . .                        | 3        |
| 3.2      | Détection des cliques par Bron-Kerbosch . . . . .            | 3        |
| 3.2.1    | Fonction detecter_cliques(G, taille_min) . . . . .           | 3        |
| 3.3      | Interprétation criminologique des deux algorithmes . . . . . | 3        |

# 1 Introduction

Deux algorithmes fondamentaux structurent ce projet. Le premier assure le nettoyage du graphe brut. Le second réalise la classification des individus par coloration et détection de cliques.

## 2 Algorithme 1 : Nettoyage du graphe

### 2.1 Objectif

Transformer un graphe brut potentiellement sale (boucles, multi-arêtes, non connexe) en un graphe propre exploitable pour l'analyse criminologique.

### 2.2 Fonction trouver\_composantes(G)

Cette fonction identifie toutes les composantes connexes du graphe.

[H] Détection des composantes connexes **Input** :  $G$  : un graphe avec un ensemble de sommets et d'arêtes

**Output** : Une liste d'ensembles contenant les sommets de chaque composante

$visites \leftarrow \emptyset$   $composantes \leftarrow []$

**pour** chaque depart dans  $G.nodes()$  **faire**

**si** depart  $\in$  visites **alors**

**continuer**

$composante \leftarrow \emptyset$   $file \leftarrow [depart]$

**tant que** file  $\neq []$  **faire**

$sommet \leftarrow file.pop(0)$

**si** sommet  $\in$  visites **alors**

**continuer**

        visites.add(sommet)  $composante.add(sommet)$

**pour** chaque voisin dans  $G.neighbors(sommet)$  **faire**

**si** voisin  $\notin$  visites **alors**

$file.append(voisin)$

$composantes.append(composante)$

**return** composantes

### 2.3 Fonction nettoyage(aretes, N)

Cette fonction est le cœur de l'algorithme 1. Elle nettoie le graphe et le rend connexe.

[H] Nettoyage du graphe **Input** : aretes : liste des arêtes brutes,  $N$  : nombre de sommets

**Output** : Un graphe  $G$  propre et connexe

$G \leftarrow$  nouveau graphe vide  $G.add\_nodes([0, 1, \dots, N - 1])$

Étape 1 : Suppression des boucles et des multi-arêtes **pour** chaque  $(u, v)$  dans aretes **faire**

**si**  $u \neq v$  **alors**

$G.add\_edge(u, v)$

Étape 2 : Détection des composantes  $composantes \leftarrow$  trouver\_composantes( $G$ )

Étape 3 : Rétablissement de la connexité **si** len(composantes)  $> 1$  **alors**

$noeud\_ref \leftarrow$  premier élément de composantes[0] **pour** chaque composante dans composantes[1 :

    ] **faire**

$noeud\_comp \leftarrow$  premier élément de composante  $G.add\_edge(noeud\_ref, noeud\_comp)$

**return**  $G$

### 2.4 Propriétés du graphe nettoyé

Après exécution de l'algorithme 1, le graphe  $G'$  obtenu possède les propriétés suivantes :

- $G'$  est connexe
- $G'$  ne contient aucune boucle
- $G'$  ne contient aucune arête multiple

### 3 Algorithmes 2 : Classification du graphe

#### 3.1 Coloration gloutonne

##### 3.1.1 Fonction colorier\_graphe(G)

Cette fonction attribue une couleur à chaque sommet en respectant la contrainte que deux sommets adjacents n'ont pas la même couleur.

[H] Coloration gloutonne par degré décroissant **Input** :  $G$  : un graphe propre

**Output** : Un dictionnaire associant à chaque sommet sa couleur

*Tri des sommets par degré décroissant*  $sommets\_tries \leftarrow \text{tri}(G.\text{nodes}(), \text{ordre décroissant du degré})$   
 $couleurs \leftarrow \{\}$

```

pour chaque sommet dans  $sommets\_tries$  faire
    Collecte des couleurs des voisins déjà colorés  $couleurs\_voisins \leftarrow \emptyset$ 
    pour chaque voisin dans  $G.\text{neighbors}(\text{sommet})$  faire
        si  $\text{voisin} \in couleurs$  alors
             $couleurs\_voisins.add(couleurs[\text{voisin}])$ 
    Recherche de la plus petite couleur disponible  $couleur \leftarrow 0$  tant que  $couleur \in couleurs\_voisins$  faire
         $couleur \leftarrow couleur + 1$ 
     $couleurs[\text{sommet}] \leftarrow couleur$ 
return  $couleurs$ 

```

#### 3.2 Détection des cliques par Bron-Kerbosch

##### 3.2.1 Fonction detecter\_cliques(G, taille\_min)

Cette fonction trouve toutes les cliques maximales de taille supérieure ou égale à un seuil donné.

[H] Bron-Kerbosch : recherche des cliques maximales **Input** :  $G$  : graphe,  $taille\_min$  : taille minimale de clique à conserver

**Output** :  $cliques$  : liste des cliques trouvées,  $appartenance$  : dictionnaire

$toutes\_cliques \leftarrow []$

**Fonction**  $BronKerbosch(R, P, X)$

```

    //  $R$  : clique en construction,  $P$  : candidats,  $X$  : exclus
    si  $P = \emptyset$  et  $X = \emptyset$  alors
        si  $\text{len}(R) \geq taille\_min$  alors
             $toutes\_cliques.append(\text{tri}(R))$ 
        return
    pour chaque sommet dans une copie de  $P$  faire
         $voisins \leftarrow$  ensemble des voisins de  $\text{sommet}$   $BronKerbosch(R \cup \{\text{sommet}\}, P \cap \text{voisins}, X \cap \text{voisins})$ 
         $P \leftarrow P \setminus \{\text{sommet}\}$   $X \leftarrow X \cup \{\text{sommet}\}$ 

```

*Lancement initial*  $BronKerbosch(\emptyset, \text{tous les sommets de } G, \emptyset)$

*Tri des cliques par taille décroissante*  $cliques \leftarrow \text{tri}(toutes\_cliques, \text{ordre décroissant de taille})$

*Calcul de l'appartenance des sommets aux cliques*  $appartenance \leftarrow \{n : [] \text{ pour } n \text{ dans } G.\text{nodes}()\}$

**pour** chaque  $i, c$  dans  $\text{énumérer}(cliques)$  **faire**

```

    pour chaque  $n$  dans  $c$  faire
         $appartenance[n].append(i)$ 

```

**return**  $(cliques, appartenance)$

#### 3.3 Interprétation criminologique des deux algorithmes

| Algorithme               | Rôle algorithmique                         | Interprétation criminologique                      |
|--------------------------|--------------------------------------------|----------------------------------------------------|
| Algorithme 1 : Nettoyage | Supprime les anomalies structurelles       | Élimine les fausses relations et relie les groupes |
| Coloration gloutonne     | Attribue des classes de sommets            | Identifie les niveaux structurels équivalents      |
| Bron-Kerbosch            | Détecte les groupes où tous se connaissent | Identifie les cellules opérationnelles soudées     |

TABLE 1 – Synthèse des deux algorithmes fondamentaux